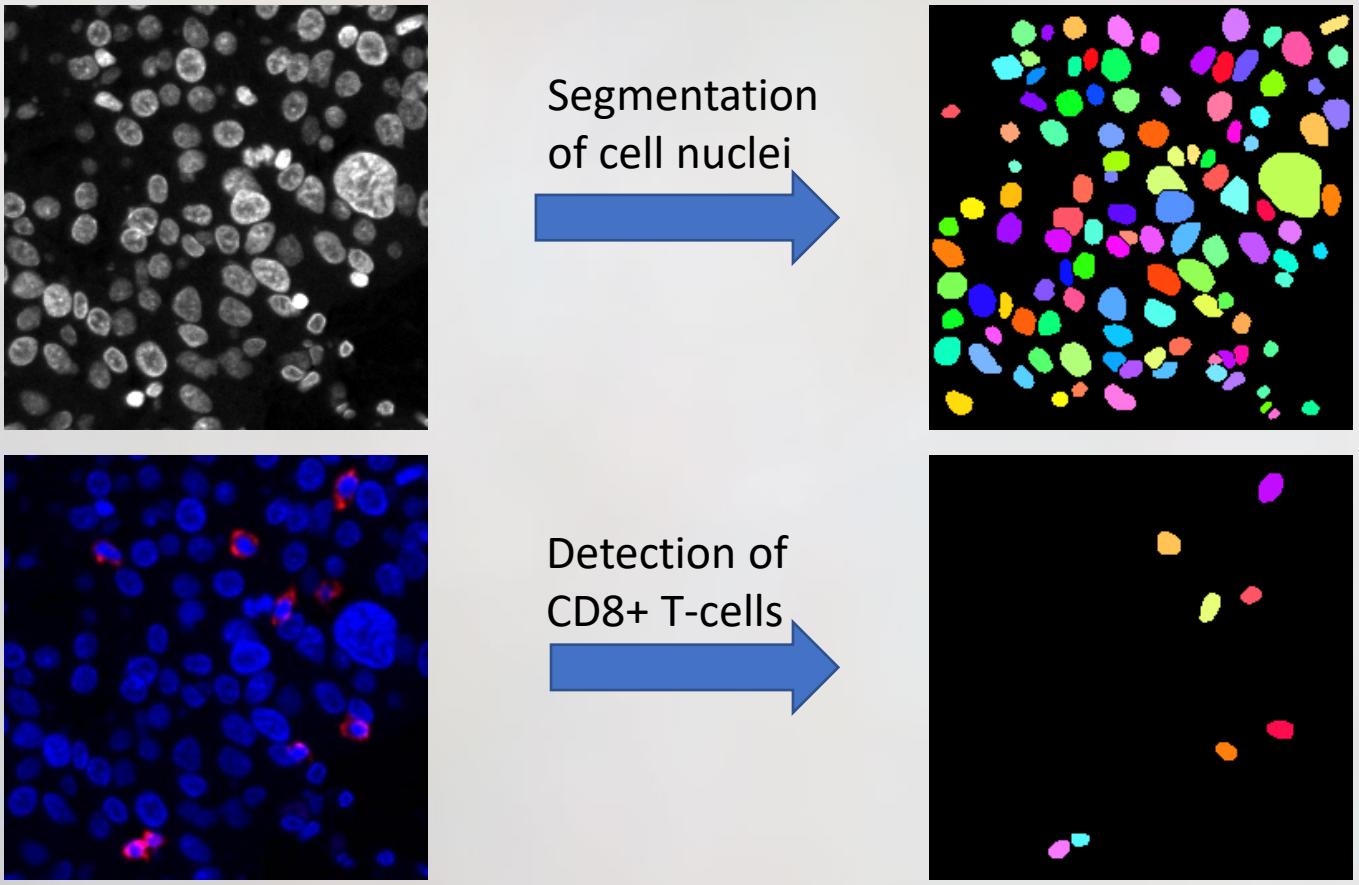
Final project

Segmentation of CD8+ lymphocytes from microscopy images of urothelial carcinoma

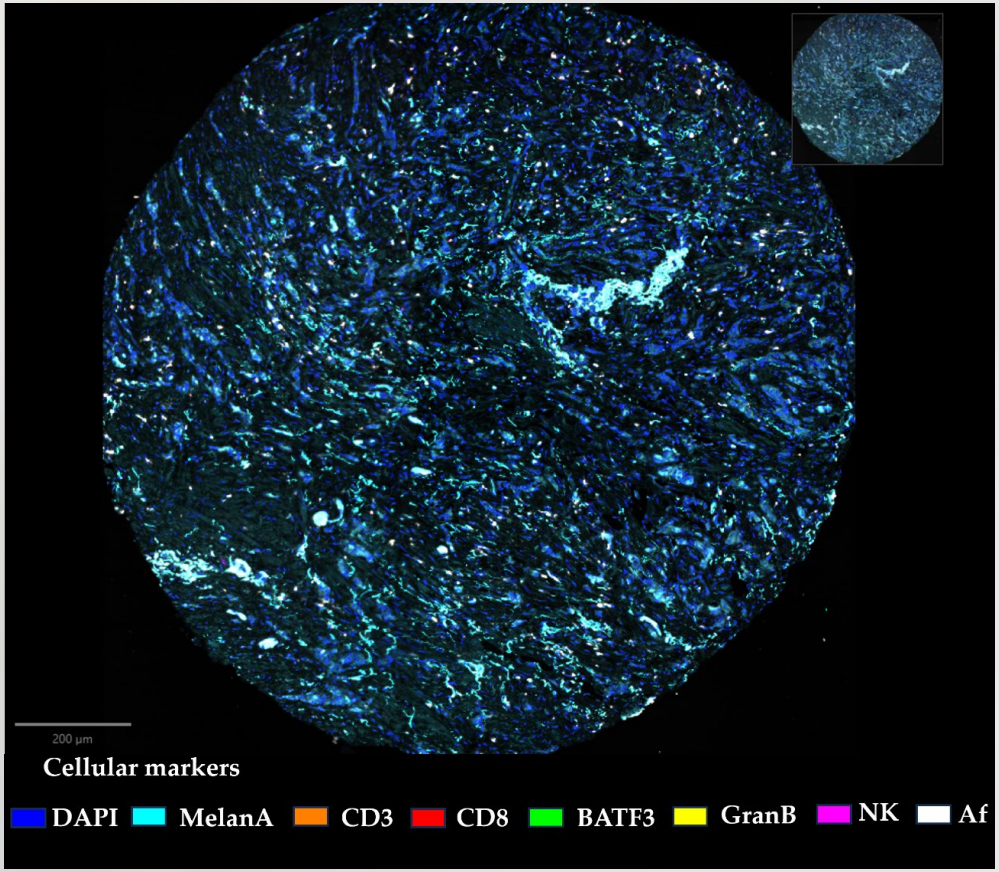
Final project

- Segmentation of CD8+ lymphocytes from microscopy images of urothelial carcinoma



Final project

- Framework: multiplex imaging for the understanding of the tumor microenvironment



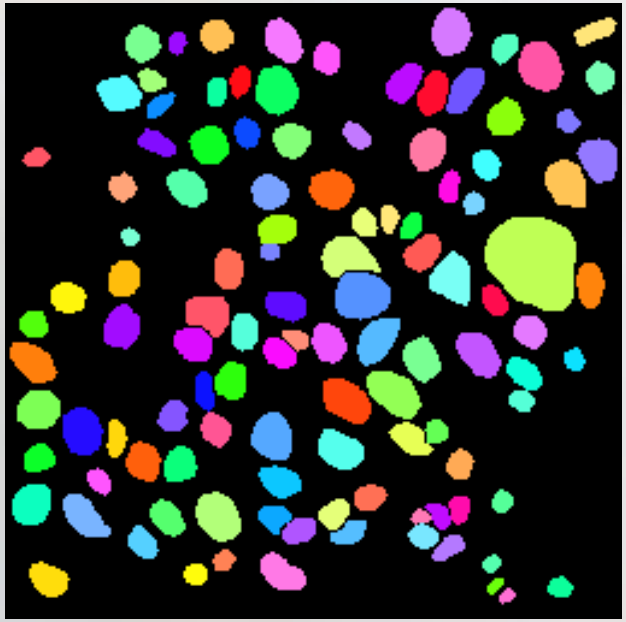
Final project

- Objectives
  - To implement a segmentation algorithm for cell nuclei in microscopy images
  - To implement a CD8+ T-cell detection algorithm
  - To evaluate the performance of the system implemented in terms of dice coefficient, detection error and execution time
- Materials
  - 21 microscopy images (256x256) of urothelial carcinoma
    - DAPI channel images containing signal from cell nuclei (DNA staining)
    - CD8 channel images for the detection of CD8+ T-cells
  - 21 labelmaps representing the ground-truth cell nuclei segmentation and 21 labelmaps of CD8+ T-cells
  - 21 RGB images with DAPI and CD8 channels fused (just for visualization!)
  - Pre-implemented functions
- Deliverables
  - Code:
    - Library `lymphocytes.py`
    - `main.py`
  - `requirements.txt`
  - Report

la parte de CD8 no tiene porque ser exactamente igual, imagenes revisadas por patologos

Final project

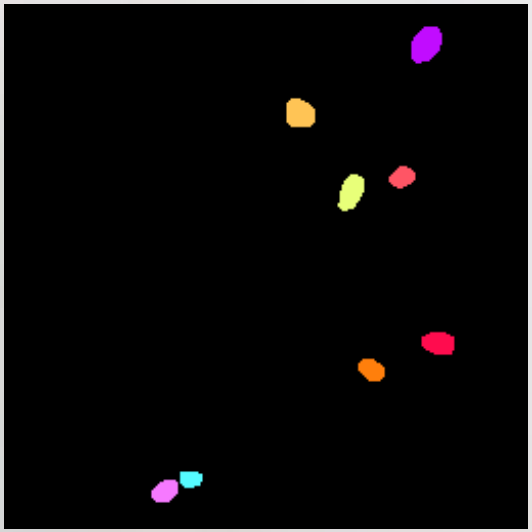
- Library `lymphocytes.py`
  - `nuclei_labelmap = Seg_Cell_Nuclei(img_DAPI, params_seg)`
    - This function is mandatory
    - `img_DAPI`: ***grayscale*** image with DAPI signal for nuclei segmentation
    - `params_seg` must be a dictionary with your parameters for segmentation



Example of nuclei segmentation

Final project

- Library `lymphocytes.py`
  - `n_CD8cells = Det_CD8_Cells(img_CD8, nuclei_labelmap, params_det)`
    - This function is mandatory
    - `img_CD8`: ***grayscale*** image with CD8 signal for CD8+ T-cell detection
    - `nuclei_labelmap`: output of previous function (labelmap of nuclei segmentation)
    - `params_det` must be a dictionary with your parameters for detection
    - `n_CD8cells`: the output number of CD8+ T-cells in the image



Example labelmap of CD8+ T-cells: 8 positive cells need to be detected for this case

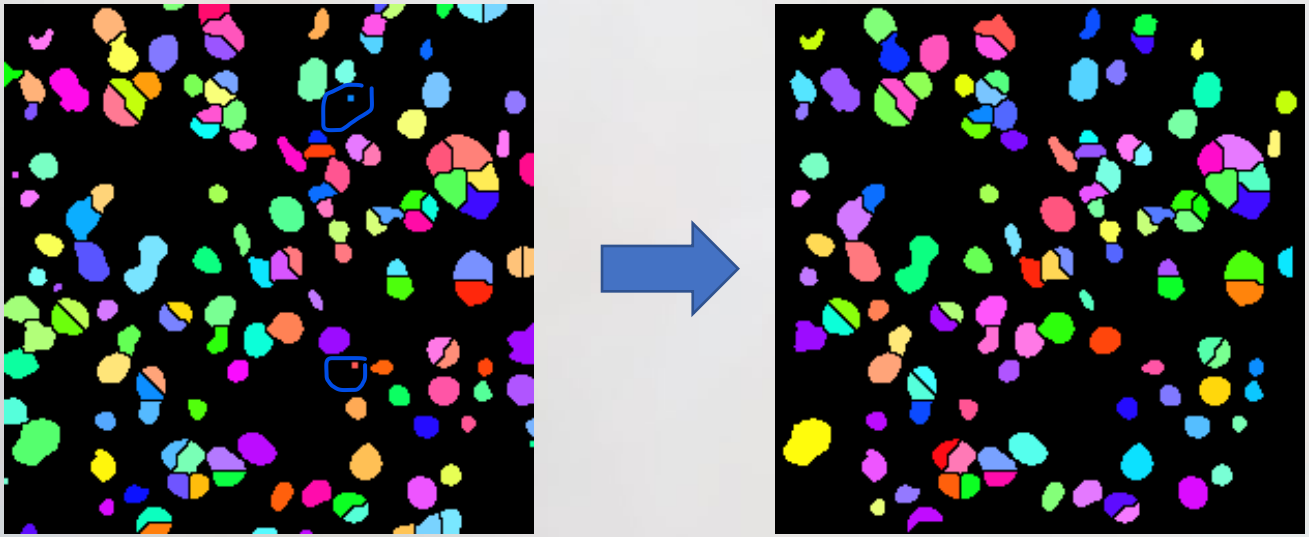
Final project

- Library `lymphocytes.py`
  - `general_results = Analyze_Set(general_configuration)`
    - This function is mandatory
    - `general_configuration` must be a dictionary with your proprietary parameters, including at least
      - `imgDAPI_path`
      - `gtNuclei_path` ground truth
      - `imgCD8_path`
      - `gtCD8_path`
      - `out_path`
    - `general_results` is a list of dictionaries with final data of the nuclei segmentation results and CD8+ T-cell detection results for each image. Each dictionary must contain these fields:
      - `image_ID`
      - `total_dice`
      - `det_error`
      - `exec_time`
    - This function will call `Seg_Cell_Nuclei()` and `Det_CD8_Cells()` functions so no image processing should be carried out in this `Analyze_Set()` function
    - The variables `params_seg` and `params_det` of the `Seg_Cell_Nuclei()` and `Det_CD8_Cells()` functions will be subsets of this variable `general_configuration`

Final project

- Optional function
  - `output_labelmap = Cleaning_Labelmap(input_labelmap, params)`
    - It would be included into the `Seg_Cell_Nuclei()` function
    - Delete incomplete nuclei that are touching the edge of the image
    - Filter out candidates by their morphological characteristics

umbral de tamaño para eliminar lo demasiado pequeño



Final project

- Script `main.py`
  - Conduct a thorough study of the performance of your code
  - What is the best dice coefficient you obtain for the nuclear segmentation?
  - What is the lowest CD8+ T-cell detection error you obtain?
  - How sensitive are the dice coefficient and detection error to the variation of parameter values?

Final project

- Supplied functions
  - `mean_dice = Evaluate_Segmentation(labelmap_test, labelmap_gt)`
    - This function evaluates the performance obtained by your algorithm in the nuclear segmentation, comparing the proposed labelmap with its corresponding ground truth
  - `det_err = Evaluate_Detection(n_CD8cells, labelmap_gt_CD8)`
    - This function evaluates the performance obtained by your algorithm in the the detection of CD8+ T-cells, comparing the proposed number of CD8+ T-cells in the image with its corresponding ground truth.
  - `fus_img, key_pressed = Fuse_Labelmap(base_img, labelmap, false_color_list, alpha, showQ, window_title)`
    - This function fuses a base image with a labelmap
- Do not forget to include the auxiliary functions into your own library

Final project

- Requirements
  - Associated with your virtual environment
- Report
  - 10 pages
  - No code at all
  - Pay special attention to the methods, results, and discussion sections
  - Performance values should be clearly stated for every setup

Final project

- Grading policy
  - Numerical results → 40%
    - Mean dice value for nuclear segmentation (20%)
    - Mean detection error of CD8+ T-cells (20%)
  - Execution time → 10%
  - Optional function → 10%
  - Report and evaluation (main.py) → 40%
  - Penalties
    - Installation problems: -1 pt
    - Sending the environment or the dataset: -2 pts
    - Not respecting the name of the functions or the format of input/output arguments.: -2 pts
    - Execution problems: up to -2 pts
- Deadline
  - 2025/01/08, 18:00